

Figure 6: Training curves for neural networks. The legend in (c) applies to all plots. For the convolutional and fully connected neural networks, the loss decreases faster using activation sampling, compared to reducing the mini-batch size further to match the memory usage. For the fully connected NN on MNIST, it is important to sample different activations for each mini-batch element, since otherwise only part of the weight vector will get updated with each iteration. For the convolutional NN on CIFAR-10, this is not an issue due to weight tying. As expected, the full memory baseline converges quicker than the low memory versions. For the RNN on Sequential-MNIST, sampling different activations at each time-step matches the performance obtained by reducing the mini-batch size.

Fraction of activations	Baseline (1.0)	0.8	0.5	0.3	0.1	0.05
ConvNet Mem	23.08	19.19	12.37	7.82	3.28	2.14
Fully Connected Mem	2.69	2.51	2.21	2.00	1.80	1.75
RNN Mem	47.93	39.98	25.85	16.43	7.01	4.66

Table 1: Peak memory (including weights) in MBytes used during training with 150 mini-batch elements. Reported values are hand calculated and represent the expected memory usage of RAD under an efficient implementation.

For the feedforward networks we tune the learning rate and ℓ_2 regularization parameter separately for each gradient estimator on a randomly held out validation set. We train with the best performing hyperparameters on bootstrapped versions of the full training set to measure variability in training. Details are in the appendix, including plots for train/test accuracy/loss, and a wider range of fraction of activations sampled. All feedforward models are trained with Adam.

In the RNN case, we also run baseline, “same sample”, “different sample” and “reduced batch” experiments. The “reduced batch” experiment used a mini-batch size of 21, while the others used a mini-batch size of 150. The learning rate was fixed at 10^{-4} for all gradient estimators, found via a coarse grid search for the largest learning rate for which optimization did not diverge. Although we did not tune the learning rate separately for each estimator, we still expect that with a fixed learning rate, the lower variance estimators should perform better. When sampling, we sample different activations at each time-step. All recurrent models are trained with SGD without momentum.

5 CASE STUDY: REACTION-DIFFUSION PDE-CONSTRAINED OPTIMIZATION

Our second application is motivated by the observation that many scientific computing problems involve a repeated or iterative computation resulting in a layered computational graph. We may apply RAD to get a stochastic estimate of the gradient by subsampling paths through the computational graph. For certain problems, we can leverage problem structure to develop a low-memory stochastic gradient estimator without exploding variance. To illustrate this possibility we consider the optimization of a linear reaction-diffusion PDE on a square domain with Dirichlet boundary conditions, representing the production and diffusion of neutrons in a fission reactor (McClarren,

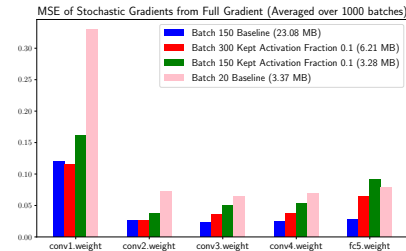


Figure 5: We visualize the gradient noise for each stochastic gradient method by computing the full gradient (over all mini-batches in the training set) and computing the mean squared error deviation for the gradient estimated by each method for each layer in the convolutional net. RAD has significantly less variance vs memory than reducing mini-batch size. Furthermore, combining RAD with an increased mini-batch size achieves similar variance to baseline 150 mini-batch elements while saving memory.

2018). Simulating this process involves solving for a potential $\phi(x, y, t)$ varying in two spatial coordinates and in time. The solution obeys the partial differential equation:

$$\frac{\partial \phi(x, y, t)}{\partial t} = D \nabla^2 \phi(x, y, t) + C(x, y, t, \theta) \phi(x, y, t)$$

We discretize the PDE in time and space and solve on a spatial grid using an explicit update rule $\phi_{t+1} = M\phi_t + \Delta t C_t \odot \phi_t$, where M summarizes the discretization of the PDE in space. The exact form is available in the appendix. The initial condition is $\phi_0 = \sin(\pi x) \sin(\pi y)$, with $\phi = 0$ on the boundary of the domain. The loss function is the time-averaged squared error between ϕ and a time-dependent target, $L = 1/T \sum_t \|\phi_t(\theta) - \phi_t^{\text{target}}\|_2^2$. The target is $\phi_t^{\text{target}} = \phi_0 + 1/4 \sin(\pi t) \sin(2\pi x) \sin(\pi y)$. The source C is given by a seven-term Fourier series in x and t , with coefficients given by $\theta \in \mathbb{R}^7$, where θ is the control parameter to be optimized. Full simulation details are provided in the appendix.

The gradient is $\frac{\partial L}{\partial \theta} = \sum_{t=1}^T \frac{\partial L}{\partial \phi_t} \sum_{i=1}^t \left(\prod_{j=i}^{t-1} \frac{\partial \phi_{j+1}}{\partial \phi_j} \right) \frac{\partial \phi_i}{\partial C_{i-1}} \frac{\partial C_{i-1}}{\partial \theta}$. As the reaction-diffusion PDE is linear and explicit, $\partial \phi_{j+1} / \partial \phi_j \in \mathbb{R}^{N_x^2 \times N_x^2}$ is known and independent of ϕ . We avoid storing C at each timestep by recomputing C from θ and t . This permits a low-memory stochastic gradient estimate without exploding variance by sampling from $\partial L / \partial \phi_t \in \mathbb{R}^{N_x^2}$ and the diagonal matrix $\partial \phi_i / \partial C_{i-1}$, replacing $\frac{\partial L}{\partial \theta}$ with the unbiased estimator

$$\sum_{t=1}^T \mathcal{S}_{P\phi_t} \left[\frac{\partial L}{\partial \phi_t} \right] \sum_{i=1}^t \left(\prod_{j=i}^{t-1} \frac{\partial \phi_{j+1}}{\partial \phi_j} \right) \mathcal{S}_{P\phi_{i-1}} \left[\frac{\partial \phi_i}{\partial C_{i-1}} \right] \frac{\partial C_{i-1}}{\partial \theta}. \quad (8)$$

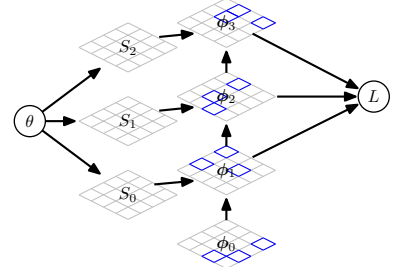
This estimator can reduce memory by as much as 99% without harming optimization; see Figure 7b.

6 RELATED WORK

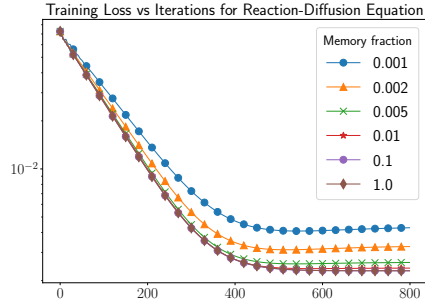
Approximating gradients and matrix operations Much thought has been given to the approximation of general gradients and Jacobians. We draw inspiration from this literature, although our main objective is designing an unbiased gradient estimator, rather than an approximation with bounded accuracy. Abdel-Khalik et al. (2008) accelerate Jacobian accumulation via random projections, in a similar manner to randomized methods for SVD and matrix multiplication. Choromanski & Sindhvani (2017) recover Jacobians in cases where AD is not available by performing a small number of function evaluations with random input perturbations and leveraging known structure of the Jacobian (such as sparsity and symmetry) via compressed sensing.

Other work aims to accelerate neural network training by approximating operations from the forward and/or backward pass. Sun et al. (2017) and Wei et al. (2017) backpropagate sparse gradients, keeping only the top k elements of the adjoint vector. Adelman & Silberstein (2018) approximate matrix multiplications and convolutions in the forward pass of neural nets using a column-row sampling scheme similar to our subsampling scheme. Their method also reduces the computational cost of the backwards pass but changes the objective landscape.

Related are invertible and reversible transformations, which remove the need to save intermediate variables on the forward pass, as these can be recomputed on the backward pass. Maclaurin et al. (2015) use this idea for hyperparameter optimization, reversing the dynamics of SGD with momentum to avoid the expense of saving model parameters at each training iteration. Gomez et al. (2017) introduce a reversible ResNet (He et al., 2016) to avoid storing activations. Chen et al. (2018) introduce Neural ODEs, which also have constant memory cost as a function of depth.



(a) Visualization of sampling



(b) RAD curves

Figure 7: Reaction-diffusion PDE expt. (b) RAD saves up to 99% of memory without significant slowdown in convergence.

Limited-memory learning and optimization Memory is a major bottleneck for reverse-mode AD, and much work aims to reduce its footprint. Gradient checkpointing is perhaps the most well known, and has been used for both reverse-mode AD (Griewank & Walther, 2000) with general layerwise computation graphs, and for neural networks (Chen et al., 2016). In gradient checkpointing, some subset of intermediate variables are saved during function evaluation, and these are used to re-compute downstream variables when required. Gradient checkpointing achieves sublinear memory cost with the number of layers in the computation graph, at the cost of a constant-factor increase in runtime.

Stochastic Computation Graphs Our work is connected to the literature on stochastic estimation of gradients of expected values, or of the expected outcome of a stochastic computation graph. The distinguishing feature of this literature (vs. the proposed RAD approach) is that it uses stochastic estimators of an objective value to derive a stochastic gradient estimator, i.e., the forward pass is randomized. Methods such as REINFORCE (Williams, 1992) optimize an expected return while avoiding enumerating the intractably large space of possible outcomes by providing an unbiased stochastic gradient estimator, i.e., by trading computation for variance. This is also true of mini-batch SGD, and methods for training generative models such as contrastive divergence (Hinton, 2002), and stochastic optimization of evidence lower bounds (Kingma & Welling, 2013). Recent approaches have taken intractable deterministic computation graphs with special structure, i.e. involving loops or the limits of a series of terms, and developed tractable, unbiased, randomized telescoping series-based estimators for the graph’s output, which naturally permit tractable unbiased gradient estimation (Tallec & Ollivier, 2017; Beatson & Adams, 2019; Chen et al., 2019; Luo et al., 2020).

7 CONCLUSION

We present a framework for randomized automatic differentiation. Using this framework, we construct reduced-memory unbiased estimators for optimization of neural networks and a linear PDE. Future work could develop RAD formulas for new computation graphs, e.g., using randomized rounding to handle arbitrary activation functions and nonlinear transformations, integrating RAD with the adjoint method for PDEs, or exploiting problem-specific sparsity in the Jacobians of physical simulators. The randomized view on AD we introduce may be useful beyond memory savings: we hope it could be a useful tool in developing reduced-computation stochastic gradient methods or achieving tractable optimization of intractable computation graphs.

ACKNOWLEDGEMENTS

The authors would like to thank Haochen Li for early work on this project. We would also like to thank Greg Gundersen, Ari Seff, Daniel Greenidge, and Alan Chung for helpful comments on the manuscript. This work is partially supported by NSF IIS-2007278.

REFERENCES

- Marín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pp. 265–283, 2016.
- Hany S Abdel-Khalik, Paul D Hovland, Andrew Lyons, Tracy E Stover, and Jean Utke. A low rank approach to automatic differentiation. In *Advances in Automatic Differentiation*, pp. 55–65. Springer, 2008.
- Menachem Adelman and Mark Silberstein. Faster neural network training with approximate tensor operations. *arXiv preprint arXiv:1805.08079*, 2018.
- Friedrich L Bauer. Computational graphs and rounding error. *SIAM Journal on Numerical Analysis*, 11(1):87–96, 1974.
- Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153), 2018.